Automatic background removal using convolutional networks

Using *pytorch*

D. Tetu, P. Contat, D. Grasset, T. Radulescu

https://github.com/DO-2K23-26/decoupeur

Introduction - What's background removal ?

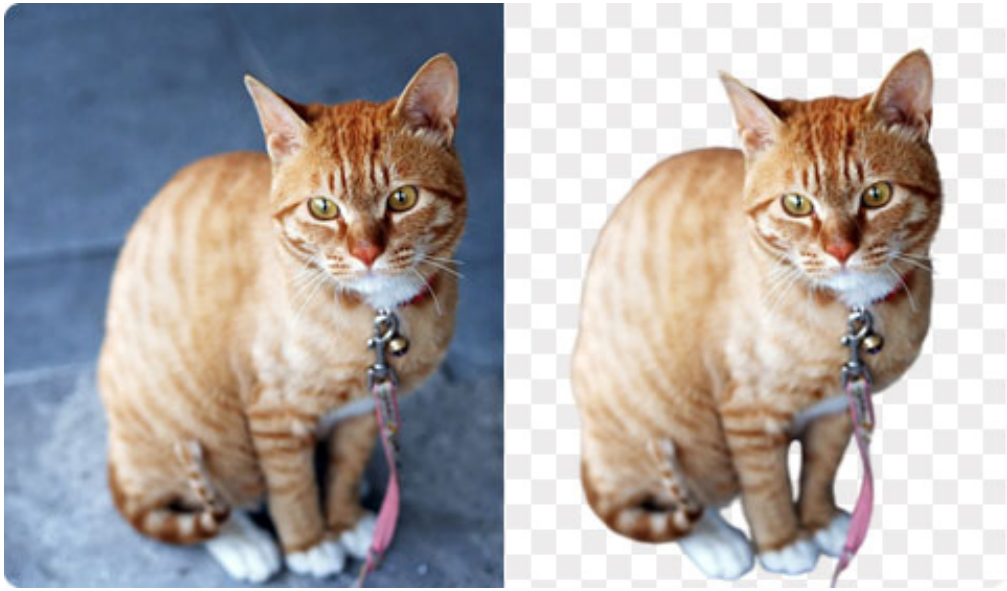

Figure 1: Foreground extraction of a cat

Goal : Isolate an element from an image as accurately as possible

Why using machine learning for that ?

- A lot of different image parameters (lighting, contrast, blurriness...).
- A lot of different poses.
- An infinity of different backgrounds.
- Almost impossible to code an algorithm for every cases.

Contents

1. Pretrained Background Remover
   - Why Transfer Learning?
   - Why U-Net? Architecture and Layers
   - Training Configuration
   - Loss Function: Dice + BCE
   - Why IoU as Evaluation Metric?
   - Quantitative Results
   - Visual Comparison: Pretrained vs From Scratch (1/2)
   - Visual Comparison: Pretrained vs From Scratch (2/2)

2. From-Scratch U-Net
   - From-Scratch U-Net: Motivation
   - Custom U-Net Architecture
   - Training Configuration (From Scratch)
   - Conclusion

Real-Time High-Resolution Background Matting [2020]

A whitepaper by students at University of Washington

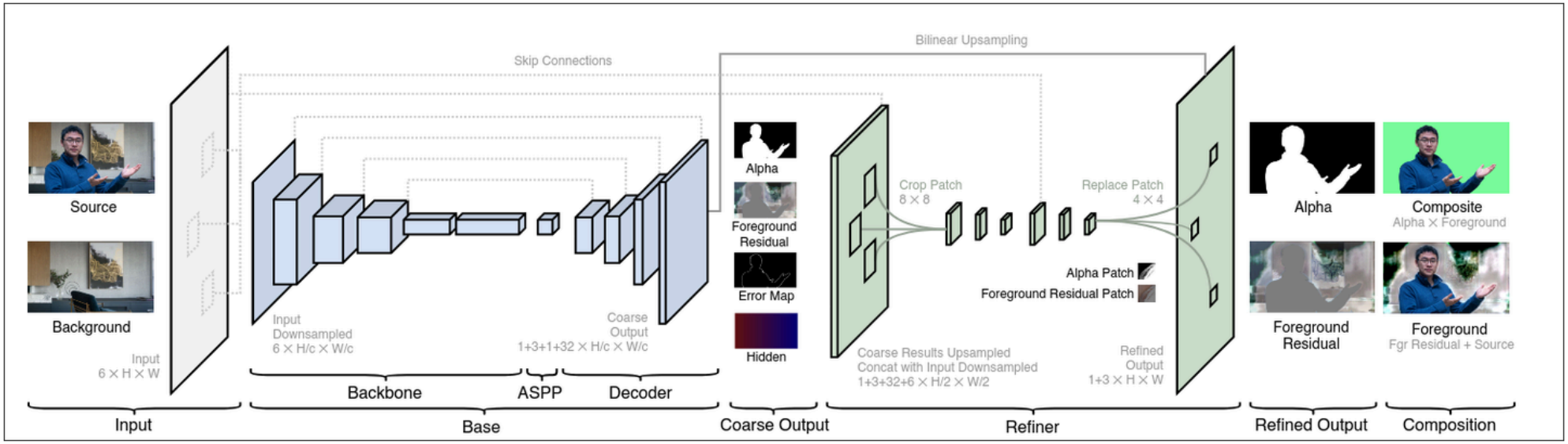

Figure 2: Architecture of their model

Pretrained Background Remover

Why Transfer Learning?

Our dataset: 34,425 labeled images of people

The problem: U-Net + ResNet34 has **24.4M parameters**. Training from scratch means the model must learn everything from zero: what an edge is, what a texture looks like, what a human silhouette is.

With only 34K images, that is not enough data to learn all of this reliably.

Transfer Learning: we reuse ResNet34 already trained on ImageNet (1.2M images). It already knows how to "see" basic visual features. We just teach it to distinguish humans from background.

Benefits:

1. Converges in 16 epochs (training took only **30 minutes** on school GPU nodes)
2. Better results on limited data
3. Lower compute cost

Why U-Net? Architecture and Layers

Goal: for each pixel of a 512x512 image, decide: human (1) or background (0)?

A standard CNN classifies a whole image into one category. U-Net is specifically designed for pixel-level segmentation: it outputs a mask of the same size as the input. This makes it the standard architecture for biomedical and human segmentation tasks.

Encoder (ResNet34, 5 stages):

1. Stage 0: 512x512 ⟶ 256x256 (64 feature maps)
2. Stage 1: 256x256 ⟶ 128x128 (64 maps)
3. Stage 2: 128x128 ⟶ 64x64 (128 maps)
4. Stage 3: 64x64 ⟶ 32x32 (256 maps)
5. Stage 4: 32x32 ⟶ 16x16 (512 maps) – bottleneck

Decoder (4 upsampling blocks):

1. 16x16 ⟶ 32x32 (256 maps)
2. 32x32 ⟶ 64x64 (128 maps)
3. 64x64 ⟶ 128x128 (64 maps)
4. 128x128 ⟶ 512x512 (32 maps) ⟶ 1 map (sigmoid)

Each decoder block uses transposed convolution to double spatial size, then

Why U-Net? Architecture and Layers

At each stage: convolutions + ReLU activations + max-pooling halve the spatial size and double the number of feature maps.

concatenates with the matching encoder stage output.

Why are they called Skip Connections?

At each level, the encoder output **skips** the bottleneck and connects directly to the corresponding decoder level. This reintroduces the fine spatial details (edges, contours) lost during compression.

Training Configuration

Setup:

1. Image size: 512x512
2. Batch size: 16
3. Epochs trained: **16**
4. Early stopping: patience = 10 (stops if validation **IoU** does not improve for 10 consecutive epochs)
5. Mixed precision (AMP): uses 16-bit floats instead of 32-bit where safe, halving memory usage and significantly speeding up GPU operations, with no accuracy loss

Optimizer: AdamW

1. Adapts learning rate per parameter automatically
2. Weight decay built-in: penalizes large weights to reduce overfitting
3. More stable convergence than plain SGD on this type of task

Scheduler: CosineAnnealingLR

1. Learning rate decreases smoothly following a cosine curve over the training run
2. Avoids large corrections near convergence, where the model is already close to optimal

Training Configuration

3. Prevents oscillating around the minimum at the end of training

Loss Function: Dice + BCE

A typical 512x512 photo of a person contains roughly 260,000 pixels of background and only about 2,000 pixels of human (less than 1%). This is called **class imbalance**: one category vastly outnumbers the other.

Binary Cross-Entropy (BCE):

1. Scores each pixel independently
2. A model predicting "all background" scores 99% accuracy
3. Completely fooled by class imbalance
4. But: stabilizes learning at the start

Dice Loss:

1. Measures the overlap between predicted mask and true mask
2. A model predicting "all background" scores 0 (no overlap)
3. Not fooled by class imbalance
4. But: slower to get started alone

Loss Function: Dice + BCE

Combined: Loss = 0.5 x BCE + 0.5 x Dice

BCE gets training started quickly. Dice ensures the model actually learns to find the human pixels. Together they work better than either alone.

Why IoU as Evaluation Metric?

IoU (Intersection over Union) measures how much the predicted mask overlaps with the true mask.

1. IoU = 1.0: prediction is perfect
2. IoU = 0.0: prediction and truth do not overlap at all
3. A model predicting all background scores IoU = 0, not 99%
4. Not fooled by class imbalance

Why not Dice then? Dice and IoU measure very similar things (Dice = 2 x IoU / (1 + IoU)), but IoU is the standard reference in every published segmentation

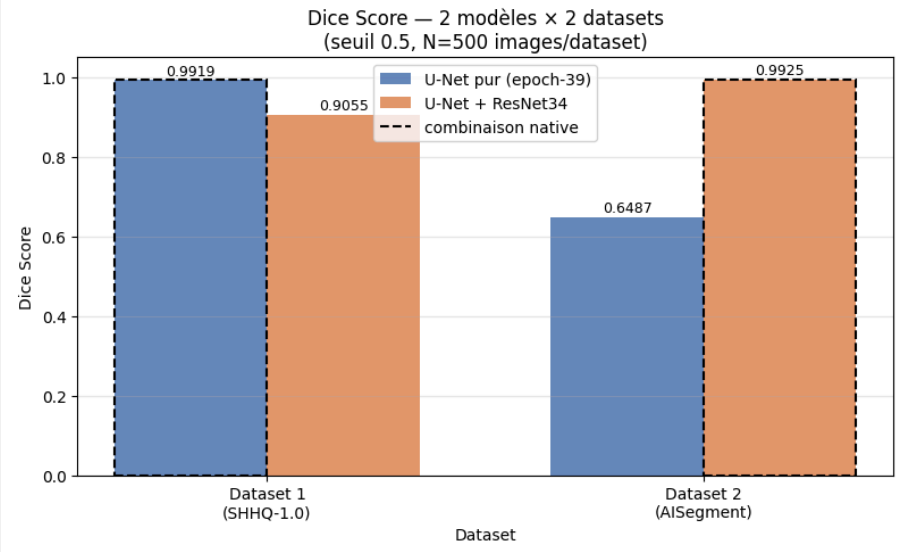

Figure 3: IoU and Dice illustrated

Why IoU as Evaluation Metric?

paper. We use Dice as the **loss** (because its gradient is better for optimization) and IoU as the **metric** (because it is the community standard for comparison).

Quantitative Results

Metrics on validation set (split: 70% train / 15% val / 15% test):

Metric	Pretrained	From Scratch
IoU	0.9856	0.9826
Dice	0.9927	0.9912
Epochs	16	40
Training time	30 min	23h

Key result

Pretrained model reaches higher IoU in fewer epochs, with lower GPU usage on school nodes.

Visual Comparison: Pretrained vs From Scratch (1/2)

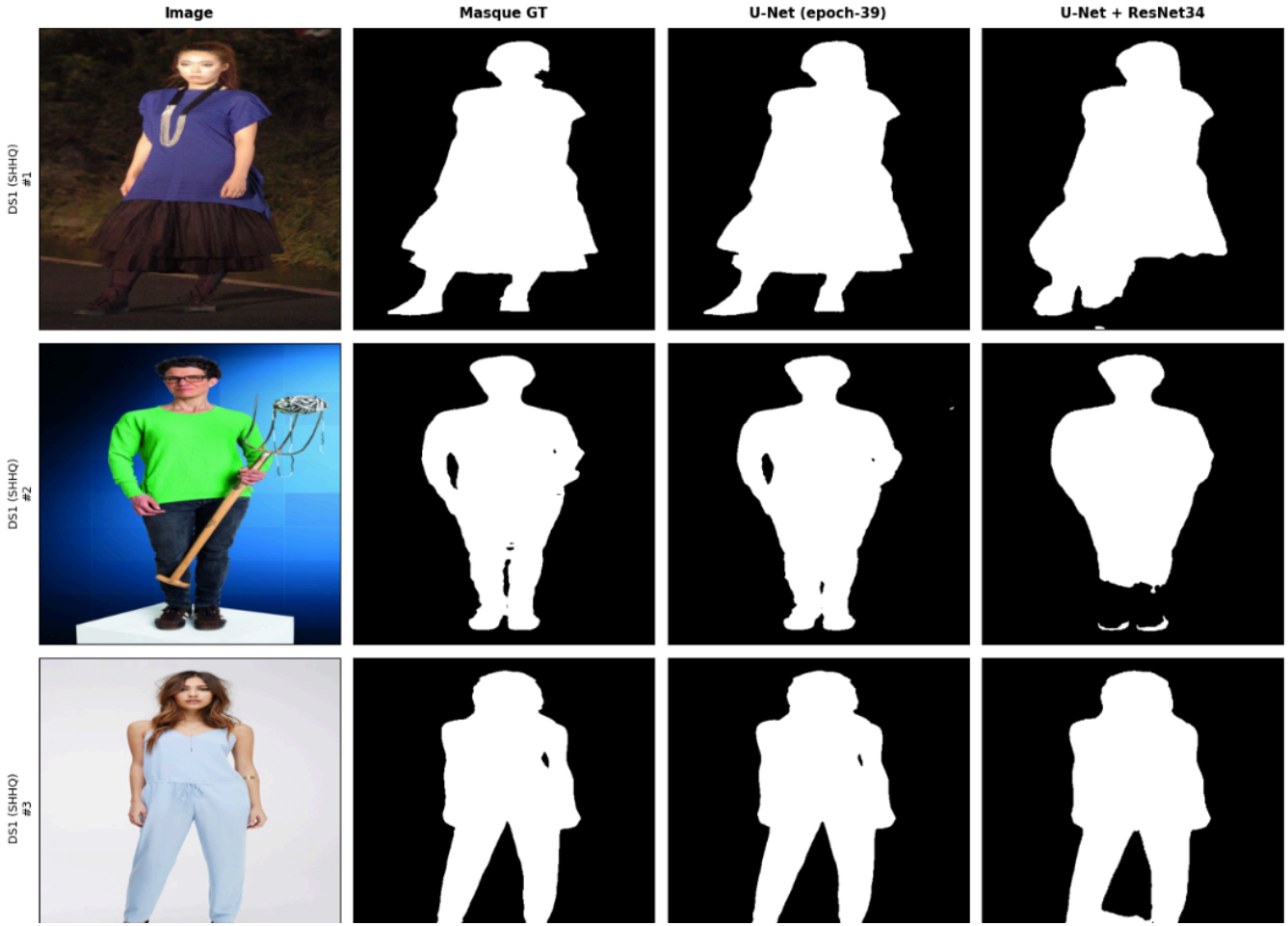

Figure 4: Predictions comparison — pretrained (top) vs from-scratch (bottom)

Visual Comparison: Pretrained vs From Scratch (2/2)

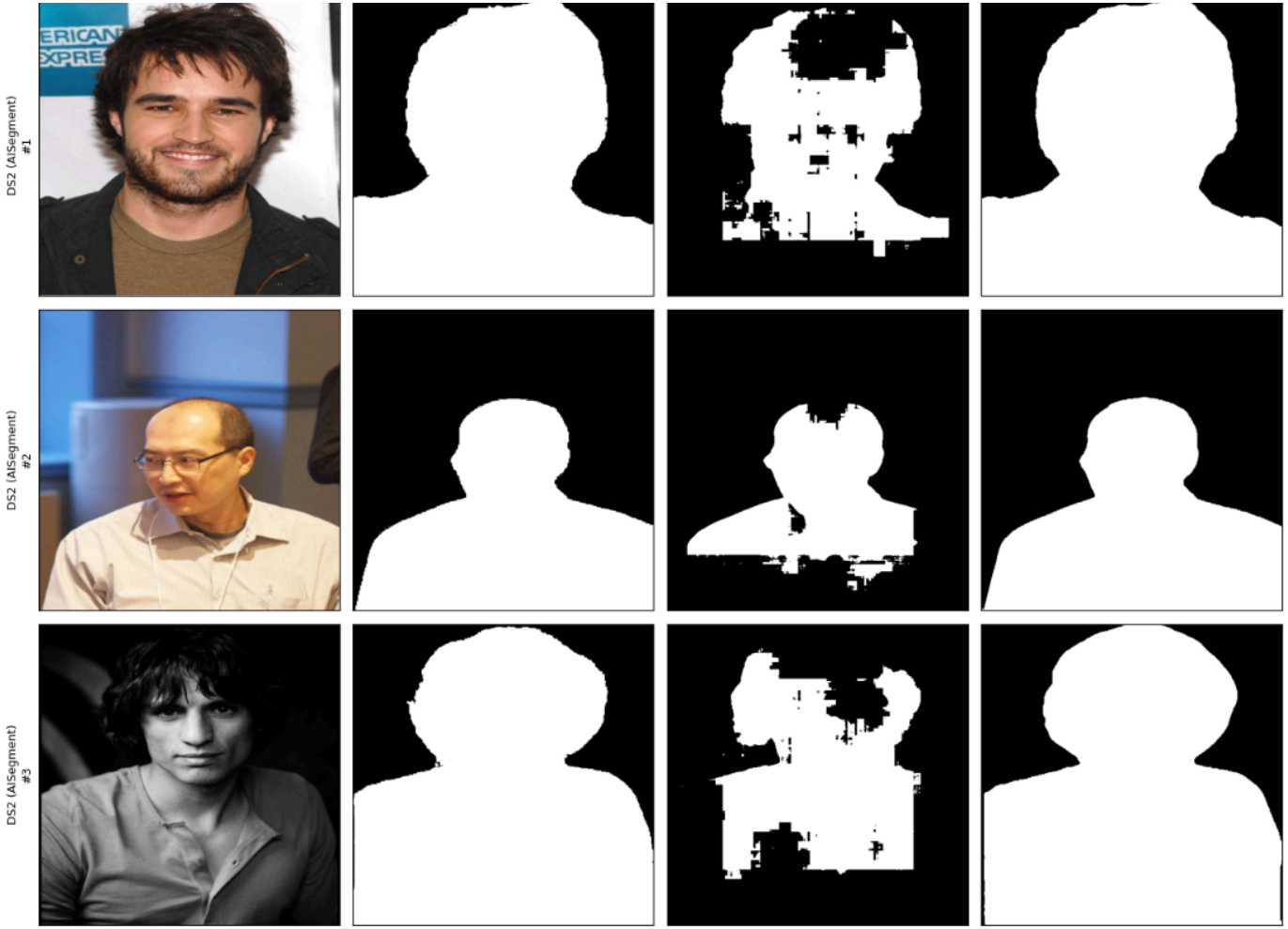

Figure 5: Predictions comparison — pretrained (top) vs from-scratch (bottom)

From-Scratch U-Net

From-Scratch U-Net: Motivation

In parallel, we trained a **custom U-Net implemented entirely from scratch** in PyTorch, without any pretrained backbone.

Goal: measure the actual contribution of transfer learning by comparing both approaches under identical conditions.

From-Scratch U-Net: Motivation

Dataset:

1. 40,000 paired image/mask samples
2. Same 512x512 resolution
3. Same augmentation pipeline

Why is this harder without pretraining?

1. The model must learn all visual features from zero
2. Edges, textures, shapes, human contours: nothing is given
3. Requires more data and more epochs to reach the same quality

Key difference vs pretrained

The pretrained model starts with 21.8M parameters already tuned on 1.2M images. The from-scratch model starts with **31M parameters** all randomly initialized, and must discover structure entirely from the training data.

Custom U-Net Architecture

Encoder (4 stages, custom ConvBlocks):

1. Input: (3, 512, 512) — RGB image
2. Stage 1: 3 ⟶ 64 feature maps, 512x512 ⟶ 256x256
3. Stage 2: 64 ⟶ 128 maps, 256x256 ⟶ 128x128
4. Stage 3: 128 ⟶ 256 maps, 128x128 ⟶ 64x64
5. Stage 4: 256 ⟶ 512 maps, 64x64 ⟶ 32x32
6. Bottleneck: 512 ⟶ 1024 maps at 16x16

Each ConvBlock = Conv2d ⟶ BatchNorm ⟶ ReLU ⟶ Conv2d ⟶ BatchNorm ⟶ ReLU

Decoder (4 upsampling blocks):

1. TransposedConv doubles spatial size at each step
2. Skip connection concatenates encoder features at each level
3. Final Conv2d (1x1) ⟶ Sigmoid ⟶ binary mask

Custom U-Net Architecture

Parameters: 31,043,521

More parameters than the pretrained model (24.4M), but all initialized randomly — which makes training harder and slower.

Training Configuration (From Scratch)

Setup:

1. Image size: 512x512
2. Batch size: 16
3. Max epochs: 50
4. Early stopping: patience = 5 (stops if **loss** does not improve for 5 consecutive epochs)
5. Checkpoint saved every epoch, training resumed automatically

Loss: BCE + Dice (0.5 / 0.5)

1. Same combined loss as the pretrained model
2. Ensures fair comparison

Optimizer: Adam

1. Adapts the learning rate per parameter automatically
2. Similar to AdamW but without built-in weight decay

Scheduler: ReduceLROnPlateau

1. Monitors the training loss
2. Halves the learning rate (factor=0.5) if no improvement after 5 epochs
3. Adapts to stagnation rather than following a fixed cosine curve

Conclusion

What we built: a background remover trained on 34K–40K images using two approaches — U-Net + pretrained ResNet34, and a custom U-Net from scratch.

Why transfer learning works:

1. ResNet34 compensates for limited data and trains in 30 min vs 23h
2. U-Net reconstructs a full-resolution mask via skip connections
3. Dice + BCE loss handles the extreme imbalance between human and background pixels
4. IoU is a meaningful metric: not fooled by the dominant background class

Results:

1. Pretrained: 0.9856 IoU in only 16 epochs (30 min)
2. From scratch: 0.9826 IoU after 40 epochs (23h)
3. Transfer learning: **3x faster convergence, same data, better score**

References